
Oliver Krauss

Modern Code - Control Structures

ModernCode | MTD.ba VZ SS25

Hagenberg 20.10.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Control Structures

- **The Problem:** Right now, our programs are linear and do only one thing
- **The Solution:** Control structures let programs **decide** and **repeat** intelligently
- **Why It Matters:** Every real program uses these → games, apps, websites, everything!
- **Control Structures:** Boolean logic → **if/else** → **switch** → loops (**while**, **for**, **do-while**)
- **Understanding:** Learning to trace code and predict what happens before it runs

Recap: Boolean Logic

- Relational: `==` `!=` `<` `<=` `>` `>=`
- Logical: `&&` (and), `||` (or), `!` (not) *short-circuiting*
- No fuzzy logic: only **boolean** in conditions
- Pitfall: assignment (`=`) vs comparison (`==`)

```
1  int hp = 15, max = 100;
2  boolean low = hp < 20;           // true
3  boolean valid = (hp > 0) && (hp <= max); // true; second check only
   if first is true
```

Recap: De Morgan & Grouping

- Negating groups: `!(A && B)` is the same as `(!A || !B)`
- Use parentheses to make intent explicit

```
1  boolean inRange = (hp >= 1) && (hp <= max);  
2  boolean outOfRange = !(hp >= 1 && hp <= max);    // clearer as:  
3  outOfRange = (hp < 1) || (hp > max);
```

The if Statement

```
1  if (hp <= 0) {  
2      IO.println("Game Over");  
3  }
```

How it works:

- Evaluate the condition: `hp <= 0`
- If **true**: execute the block `{...}`
- If **false**: skip the block entirely

Key points:

- Condition must be **boolean** (no truthy/falsy like Python/JavaScript)
- Always use braces `{...}` even for single statements
- Indent the block for readability

if-else: Two Choices

```
1  if (hp <= 0) {  
2      IO.println("Game Over");  
3  } else {  
4      IO.println("You're alive!");  
5  }
```

How it works:

- If condition is **true**: execute first block
- If condition is **false**: execute **else** block
- **Exactly one** block will execute

Common pattern: Checking for a specific condition, with a fallback action

else if: Multiple Choices I

```
1  if (hp <= 0) {  
2      IO.println("Game Over");  
3  } else if (hp < 20) {  
4      IO.println("Low Health!");  
5  } else if (hp < 50) {  
6      IO.println("Moderate Health");  
7  } else {  
8      IO.println("You're fine.");  
9  }
```

else if: Multiple Choices II

How it works:

- Check conditions in order (top to bottom)
- Execute the **first** block where condition is **true**
- If no conditions are **true**, execute **else** block
- **Only one** block executes (even if multiple conditions are true)

Order matters! Put most specific conditions first.

The Dangling else Problem

```
1  int x = 5, y = 10;
2  if (x > 0)
3      if (y > 0)
4          IO.println("Both positive");
5  else
6      IO.println("x is not positive");
```

What prints?

The Dangling else Problem

```
1  int x = 5, y = 10;
2  if (x > 0)
3      if (y > 0)
4          IO.println("Both positive");
5  else
6      IO.println("x is not positive");
```

What prints? "Both positive" (surprise!)

The Dangling else Problem II

Now try with y = -5:

```
1  int x = 5, y = -5;
2  if (x > 0)
3      if (y > 0)
4          IO.println("Both positive");
5  else
6      IO.println("x is not positive");
```

What prints?

The Dangling else Problem II

Now try with $y = -5$:

```
1  int x = 5, y = -5;
2  if (x > 0)
3      if (y > 0)
4          IO.println("Both positive");
5  else
6      IO.println("x is not positive");
```

What prints? "x is not positive" (even more surprising!)

The Dangling else Problem III

Why? The **else** belongs to the **innermost** **if**:

```
1  if (x > 0) {
2      if (y > 0) {
3          IO.println("Both positive");
4      } else {
5          // This else belongs to the inner if!
6          IO.println("x > 0 but y <= 0");
7      }
8  }
9  // No else for the outer if
```

Solution: Always use braces to make your intent clear!

switch: Multi-Way Decision I

```
1  int action = 2; // 1=Attack, 2=Heal, 3=Defend
2  switch (action) {
3      case 1: IO.println("You attack!"); break;
4      case 2: IO.println("You heal.");   break;
5      case 3: IO.println("You defend."); break;
6      default: IO.println("Unknown action");
7  }
```

switch: Multi-Way Decision II

How it works:

- Evaluate the expression: **action**
- Match against each **case** value
- Execute the matching case's code
- **break** exits the switch (prevents fall-through)
- **default** handles unmatched values

Key points:

- Expression must be **int**, **char**, **String**, or **enum**
- Cases must be compile-time constants

What Are Compile-Time Constants? I

```
1  // VALID: compile-time constants
2  final int ATTACK = 1;
3  final int HEAL = 2;
4  final int DEFEND = 3;
5
6  switch (action) {
7      case ATTACK: IO.println("Attack!"); break;
8      case HEAL:   IO.println("Heal!");   break;
9      case DEFEND: IO.println("Defend!"); break;
10 }
```


What Are Compile-Time Constants? II

```
1  // INVALID: not compile-time constants
2  int userInput = 5;
3  switch (action) {
4      case userInput:    // Error: not constant
5  }
```

Compile-time constants:

- **final** variables with literal values: **final int x = 5;**
- Literal values directly: **case 5: case (2+3):** (not recommended)

Why this restriction? The compiler needs to know all possible case values to create an efficient jump table.

switch Best Practices

- **default** is optional but recommended
- Use **break** to prevent fall-through (unless intentional)
- Group related cases together when fall-through is useful
- Consider using **final** constants for case values

switch: The Fall-Through Problem

```
1  int day = 2; // Tuesday
2  switch (day) {
3      case 1: IO.println("Monday");
4      case 2: IO.println("Tuesday");
5      case 3: IO.println("Wednesday");
6      case 4: IO.println("Thursday");
7      case 5: IO.println("Friday");
8      default: IO.println("Weekend or invalid");
9  }
```

What prints?

switch: The Fall-Through Problem

```
1  int day = 2; // Tuesday
2  switch (day) {
3      case 1: IO.println("Monday");
4      case 2: IO.println("Tuesday");
5      case 3: IO.println("Wednesday");
6      case 4: IO.println("Thursday");
7      case 5: IO.println("Friday");
8      default: IO.println("Weekend or invalid");
9  }
```

What prints? Tuesday Wednesday Thursday Friday Saturday Weekend or invalid

Why? Without **break**, execution continues through all subsequent cases!

switch: The Fall-Through Problem II

When fall-through is useful:

```
1  switch (grade) {
2      case 1:  // Sehr gut (Excellent)
3      case 2:  // Gut (Good)
4      case 3:  // Befriedigend (Satisfactory)
5      case 4:  // Genuiegend (Sufficient)
6          IO.println("Bestanden (Passed)"); break;
7      case 5:  // Nicht genuiegend (Insufficient)
8          IO.println("Nicht bestanden (Failed)"); break;
9  }
```

Why fall-through works here: Multiple case values (1, 2, 3, 4) all execute the same "Passed" code, then break. Only grade 5 gets the "Failed" message.

What Are Loops?

Loops = Repetition in Programming

Real-world examples:

- **Counting:** "Count from 1 to 10" → repeat counting 10 times
- **Checking:** "Keep asking until you get a valid answer" → repeat asking until condition is met
- **Processing:** "Do this for every item in a list" → repeat for each item

Why loops matter:

- Without loops: write the same code 100 times
- With loops: write it once, tell the computer to repeat it 100 times

The three questions every loop answers:

1. **When to start?** (initialization)
2. **When to continue?** (condition)
3. **How to move forward?** (update)

Loops

while
(pre-test)

When to use: Unknown number of iterations

Example: Game loop, input validation

for
(counted)

When to use: Known number of iterations

Example: Array processing, counting

do-while
(post-test)

When to use: Must run at least once

Example: Menu systems, user input

Key Questions for Choosing:

- Do you know how many times to repeat? → **for**
- Must the loop run at least once? → **do-while**
- Otherwise? → **while**

while: Pre-Test Loop

```
1  boolean done = false; int turn = 0;
2  while (!done) {
3      IO.println("Turn " + turn);
4      if (turn == 5) done = true;    // update condition
5      turn++;
6  }
```

How while works:

- **Pre-test:** Check condition **before** entering the loop
- **Keep looping:** As long as condition is **true**
- **Exit when:** Condition becomes **false**

What is Tracing / Dry-Running?

Tracing = Reading code like a computer does

What you do:

- **Read line by line:** Follow the code execution step by step
- **Track variables:** Write down how values change
- **Simulate output:** Figure out what gets printed
- **Predict flow:** See where the program goes next

Why it's powerful:

- **Find bugs:** See exactly where things go wrong
- **Understand logic:** Figure out how complex code works
- **Debug without running:** Catch errors on paper first
- **Improve code:** Spot inefficiencies and better approaches

Getting AI Help with Dry Runs

How to ask AI for tracing help:

Good prompts:

- **"Trace this code step by step:"** + paste your code
- **"Show me a trace table for:"** + specific loop/function
- **"What's the output of this code?"** + explain what you expect
- **"Help me debug this:"** + describe the problem you're seeing

What to include:

- **Your code:** Complete, runnable code snippet
- **Your question:** What you want to understand
- **Your attempt:** What you've tried so far
- **Expected vs actual:** What you think should happen vs what does

Remember: AI is a tool to help you learn, not replace your thinking!

Trace a while-loop: Game Example

```
1    int playerHealth = 10;
2    int enemyDamage = 5;
3    while (playerHealth > 0) {
4        IO.println("Player health: " + playerHealth);
5        playerHealth = playerHealth - enemyDamage;
6        IO.println("Took " + enemyDamage + " damage!");
7    }
8    IO.println("Game Over!");
9
```

Let's trace this step by step!

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"
3	5	true	-

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"
3	5	true	-
4	5	-	"health: 5"

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"
3	5	true	-
4	5	-	"health: 5"
5	0	-	-

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"
3	5	true	-
4	5	-	"health: 5"
5	0	-	-
6	0	-	"damage!"

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"
3	5	true	-
4	5	-	"health: 5"
5	0	-	-
6	0	-	"damage!"
3	0	false	(exit loop)

Trace a while-loop: Step-by-Step

```
1  int playerHealth = 10;
2  int enemyDamage = 5;
3  while (playerHealth > 0) {
4      IO.println("Player health: "
5          + playerHealth);
6      playerHealth = playerHealth -
7          enemyDamage;
8      IO.println("Took " +
9          enemyDamage + " damage!");
10 }
11 IO.println("Game Over!");
```

Trace table:

Line	Health	> 0?	Output
1	10	-	-
2	10	-	-
3	10	true	-
4	10	-	"health: 10"
5	5	-	-
6	5	-	"damage!"
3	5	true	-
4	5	-	"health: 5"
5	0	-	-
6	0	-	"damage!"
3	0	false	(exit loop)
8	0	-	"Game Over"

The Bug: Infinite Loop! I

```
1    int playerHealth = 10;
2    int enemyDamage = 5;
3
4    while (playerHealth > 0) {
5        IO.println("Player health: " + playerHealth);
6        // BUG: Missing update!
7        // playerHealth = playerHealth - enemyDamage;
8        IO.println("Took " + enemyDamage + " damage!");
9    }
10
```

What happens now?

The Bug: Infinite Loop! II

- **playerHealth** stays 10 forever
- **10 > 0** is always **true**
- Loop never exits!
- Program hangs indefinitely

The fix: Always ensure the loop condition can become **false**!

Common causes:

- Missing update statement
- Update doesn't affect the condition
- Wrong boundary condition (< vs <=)

for: Counted Loop

```
1  for (INITIALIZATION; CONDITION; UPDATE) {  
2      BODY  
3  }
```

```
1  for (int i = 0; i < 5; i++) {  
2      IO.println("Frame " + i);  
3  }
```

How for works:

- **Initialization:** `int i = 0` (runs once at start)
- **Condition:** `i < 5` (checked before each iteration)
- **Update:** `i++` (runs after each iteration)

for: Countdown

```
1  for (int count = 3; count > 0; count--) {  
2      IO.println("Countdown: " + count);  
3  }
```

Execution flow:

for: Countdown

```
1  for (int count = 3; count > 0; count--) {  
2      IO.println("Countdown: " + count);  
3  }
```

Execution flow:

-- Start: **count** = 3, check **3 > 0** → true, print "Countdown: 3"

for: Countdown

```
1  for (int count = 3; count > 0; count--) {  
2      IO.println("Countdown: " + count);  
3  }
```

Execution flow:

- Start: **count** = 3, check **3 > 0** → true, print "Countdown: 3"
- Update: **count** = 2, check **2 > 0** → true, print "Countdown: 2"

for: Countdown

```
1  for (int count = 3; count > 0; count--) {  
2      IO.println("Countdown: " + count);  
3  }
```

Execution flow:

- Start: **count** = 3, check $3 > 0 \rightarrow \text{true}$, print "Countdown: 3"
- Update: **count** = 2, check $2 > 0 \rightarrow \text{true}$, print "Countdown: 2"
- Update: **count** = 1, check $1 > 0 \rightarrow \text{true}$, print "Countdown: 1"

for: Countdown

```
1  for (int count = 3; count > 0; count--) {  
2      IO.println("Countdown: " + count);  
3  }
```

Execution flow:

- Start: **count** = 3, check $3 > 0 \rightarrow \text{true}$, print "Countdown: 3"
- Update: **count** = 2, check $2 > 0 \rightarrow \text{true}$, print "Countdown: 2"
- Update: **count** = 1, check $1 > 0 \rightarrow \text{true}$, print "Countdown: 1"
- Update: **count** = 0, check $0 > 0 \rightarrow \text{false}$, exit loop

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

-- Start: `i = 0`, check `0 <= 20` → true, print "Multiple of 5: 0"

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

- Start: `i = 0`, check `0 <= 20` → true, print "Multiple of 5: 0"
- Update: `i = 5`, check `5 <= 20` → true, print "Multiple of 5: 5"

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

- Start: $i = 0$, check $0 \leq 20 \rightarrow \text{true}$, print "Multiple of 5: 0"
- Update: $i = 5$, check $5 \leq 20 \rightarrow \text{true}$, print "Multiple of 5: 5"
- Update: $i = 10$, check $10 \leq 20 \rightarrow \text{true}$, print "Multiple of 5: 10"

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

- Start: **i** = 0, check **0** <= 20 → true, print "Multiple of 5: 0"
- Update: **i** = 5, check **5** <= 20 → true, print "Multiple of 5: 5"
- Update: **i** = 10, check **10** <= 20 → true, print "Multiple of 5: 10"
- Update: **i** = 15, check **15** <= 20 → true, print "Multiple of 5: 15"

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

- Start: **i** = 0, check **0** <= 20 → true, print "Multiple of 5: 0"
- Update: **i** = 5, check **5** <= 20 → true, print "Multiple of 5: 5"
- Update: **i** = 10, check **10** <= 20 → true, print "Multiple of 5: 10"
- Update: **i** = 15, check **15** <= 20 → true, print "Multiple of 5: 15"
- Update: **i** = 20, check **20** <= 20 → true, print "Multiple of 5: 20"

for: Update does not need to be ++ or --

```
1  for (int i = 0; i <= 20; i += 5) {  
2      IO.println("Multiple of 5: " + i);  
3  }
```

Execution flow:

- Start: `i = 0`, check `0 <= 20` → true, print "Multiple of 5: 0"
- Update: `i = 5`, check `5 <= 20` → true, print "Multiple of 5: 5"
- Update: `i = 10`, check `10 <= 20` → true, print "Multiple of 5: 10"
- Update: `i = 15`, check `15 <= 20` → true, print "Multiple of 5: 15"
- Update: `i = 20`, check `20 <= 20` → true, print "Multiple of 5: 20"
- Update: `i = 25`, check `25 <= 20` → false, exit loop

do-while: Run At Least Once

```
1  int choice;
2  do {
3      IO.println("1) Attack  2) Heal  0) Quit");
4      choice = 1; // simulate input
5  } while (choice != 0);
```

How do-while works:

- **Post-test:** Condition checked **after** the loop body
- **Always runs once:** Body executes before first condition check
- **Keep looping:** As long as condition is **true**
- **Exit when:** Condition becomes **false**

Loop Control: break

```
1  for (int t = 0; t < 10; t++) {  
2      if (t == 7) {  
3          IO.println("Found 7, stopping early!");  
4          break; // Exit the loop immediately  
5      }  
6      IO.println("Processing: " + t);  
7  }  
8  IO.println("Loop finished");
```

How break works:

- **Immediate exit:** Jumps out of the loop completely
- **No more iterations:** Loop condition is ignored
- **Execution continues:** After the loop body

Loop Control: continue

```
1  for (int t = 0; t < 10; t++) {  
2      if (t % 2 == 1) {  
3          IO.println("Skipping odd number: " + t);  
4          continue; // Skip to next iteration  
5      }  
6      IO.println("Processing even number: " + t);  
7  }
```

How continue works:

- **Skip current iteration:** Jumps to the loop update
- **Loop continues:** Next iteration starts normally
- **Body skipped:** Code after continue doesn't run

Loop Control: When to Use break & continue

break examples:

- **Search loops:** Found the item, stop looking
- **Game loops:** Player won/lost, end the game
- **Error handling:** Something went wrong, stop processing
- **Input validation:** Got valid input, no need to ask again

continue examples:

- **Data filtering:** Skip invalid or unwanted items
- **Processing:** Skip items that don't need full processing
- **Error recovery:** Skip problematic items, continue with others

Real-world analogy:

- **break:** Like finding your keys and stopping the search
- **continue:** Like skipping full parking spots while looking for an empty one

Loop Control: Best Practices

When to use **break** & **continue**:

- **break:** When you need to exit early (found target, error occurred)
- **continue:** When you want to skip current iteration but continue looping

When **NOT** to use them:

- **Complex logic:** Makes code harder to read and debug
- **Multiple levels:** Can make nested loops confusing
- **Overuse:** Can indicate poor loop design

Better alternatives:

- **Clear conditions:** Design loops with natural exit conditions
- **Boolean flags:** Use variables to control loop behavior
- **Separate functions:** Extract complex logic into helper methods

Remember: These are tools, not crutches. Use them when they make code clearer!

Common Mistakes & Debugging Tips I

Infinite Loops

The most common loop mistake: the condition never becomes false.

```
1  int count = 0;
2  while (count < 5) {
3      IO.println("Count: " + count);
4      // BUG: Missing count++ update!
5  }
```

Why it's infinite: `count` stays 0 forever, so `0 < 5` is always true.

Common Mistakes & Debugging Tips II

Fix: Always ensure your loop condition can become false!

```
1  int count = 0;
2  while (count < 5) {
3      IO.println("Count: " + count);
4      count++; // FIXED: Update the variable
5  }
```

Common Mistakes & Debugging Tips III

Assignment vs Comparison

Irrelevant for Java, but many other languages (Python, JS) have this issue:

Using `=` (assignment) instead of `==` (comparison) in conditions.

```
1  int health = 100;
2  if (health = 0) { // BUG: assigns 0 to health, always true!
3      IO.println("Game Over");
4  }
```

Why it's wrong: `health = 0` assigns 0 to health and returns 0, which is false, but the assignment happened!

Common Mistakes & Debugging Tips IV

Fix: Use `==` for comparison, `=` for assignment.

```
1  int health = 100;
2  if (health == 0) { // FIXED: compares health to 0
3      IO.println("Game Over");
4  }
```

Common Mistakes & Debugging Tips V

Off-by-One Errors

Loops that run one time too many or too few due to boundary conditions.

```
1  // Want to print numbers 1 to 5
2  for (int i = 1; i <= 5; i++) { // CORRECT: includes 5
3      IO.println(i);
4  }
5  // Common mistake: using < instead of <=
6  for (int i = 1; i < 5; i++) { // BUG: prints 1,2,3,4
7      IO.println(i);
8  }
```


Common Mistakes & Debugging Tips VI

Complex Conditions that are hard to read and debug.

```
1  if (health > 0 && (damage < maxDamage || armor > 0) && !isDead) {
2      // What does this even mean?
3  }
4  // BETTER: Break it down with variables
5  boolean hasHealth = health > 0;
6  boolean hasProtection = damage < maxDamage || armor > 0;
7  boolean isAlive = !isDead;
8  if (hasHealth && hasProtection && isAlive) {
9      // Much clearer what we're checking!
10 }
```

Common Mistakes & Debugging Tips VII

Debugging tip: Print your boolean variables to see what's happening!

```
1  IO.println("hasHealth: " + hasHealth);  
2  IO.println("hasProtection: " + hasProtection);  
3  IO.println("isAlive: " + isAlive);
```

Summary: Control Structures

Conditional Statements

- **if**: Single condition execution
- **if-else**: Two-way branching
- **if-else if-else**: Multi-way decisions
- Dangling else problem: Always use braces!

Switch Statements

- Multi-way decisions with **switch/case**
- **break** prevents fall-through
- **default** handles unmatched cases
- Cases must be compile-time constants

Loops

- **while**: Pre-test, unknown iterations
- **for**: Counted, known iterations
- **do-while**: Post-test, runs at least once

Loop Control

- **break**: Exit loop early
- **continue**: Skip current iteration

Key Skills

- Tracing/dry-running code execution
- Avoiding infinite loops and off-by-one errors
- Writing readable, maintainable conditions
- Debugging with print statements